Using Linux Commands to Manage File Permissions

Project description

As a security professional supporting a research team, I audited an existing Linux file system to confirm that file and directory permissions matched the access the team was actually authorized to have. Working in the researcher2 user's projects directory, I used ls -la to check current permissions on all files, including hidden ones, compared them against the organization's access policy, and used chmod to correct any permissions that granted broader access than intended — removing unauthorized write access on a shared file, locking down an archived hidden file to read-only for the owner and group, and restricting a drafts directory so that only its owner, researcher2, can access it.

Check file and directory details

To check the current permissions on the files and subdirectories inside the projects directory — including hidden files, which don't show up with a plain ls — I used ls -la:

```
researcher2@Linux-Instance:~/projects$ ls -la
```

The `-l` flag produces the “long listing” format, which shows the full permission string, owner, group, size, and modification date for each item, instead of just the file names. The `-a` flag tells ls to show all entries, including hidden files and directories (any name starting with a dot, like .project_x.txt) that would otherwise be left out of the listing.

The output of this command was:

```
total 32
drwxrwxr-x 4 researcher2 research 4096 Aug 20 09:14 .
drwxr-xr-x 6 researcher2 research 4096 Aug 18 11:02 ..
-rw-rw-rw- 1 researcher2 research  842 Aug 19 14:22 lab_notes.txt
-rwxrwxrwx 1 researcher2 research  128 Aug 20 09:10 .project_x.txt
drwxrwxr-x 2 researcher2 research 4096 Aug 19 16:45 drafts
```

Comparing this against the organization's policy, two problems stood out: lab_notes.txt and the hidden file .project_x.txt both grant write access to “other” (anyone on the system), which the organization does not allow, and the drafts directory is readable and executable by the whole research group, when only researcher2 should be able to access it at all.

Describe the permissions string

Using lab_notes.txt as an example, its permission string from the ls -la output above is -rw-rw-rw-. This 10-character string represents the file's type and its full set of read, write, and execute permissions for the file's owner, group, and everyone else (“other”):

Character(s)	Segment	Value	Meaning	
1	File type	-	Regular file (a leading "d" would mean directory, "l" a symlink)	

2-4	Owner (user) permissions	rw-	Owner can read and write, but not execute
5-7	Group permissions	rw-	Group members can read and write, but not execute
8-10	Other permissions	rw-	Everyone else can read and write, but not execute

Reading the string left to right, it breaks into a 1-character file-type flag followed by three 3-character permission groups — owner, group, then other — each in the fixed order read (r), write (w), execute (x), with a dash standing in for any permission that isn't granted. -rw-rw-rw- means this is a regular file that any user on the system — not just the owner or group — can currently read from and write to, which is exactly the kind of over-permissive access this activity is meant to catch and correct.

Change file permissions

Based on the permissions in the previous step, lab_notes.txt needs to be corrected: it currently allows “other” to write to it (-rw-rw-rw-), which violates the organization's rule that no one outside the file's owner and group should have write access to any file. I used chmod to remove the write bit for other:

```
researcher2@Linux-Instance:~/projects$ chmod o-w lab_notes.txt
researcher2@Linux-Instance:~/projects$ ls -l lab_notes.txt
-rw-rw-r-- 1 researcher2 research 842 Aug 19 14:22 lab_notes.txt
```

The `chmod` (change mode) command modifies a file's permissions. The symbolic argument `o-w` targets the “other” permission group specifically (`o`) and removes (`-`) the write permission (`w`) from it, without touching the owner's or group's permissions. Running ls -l afterward confirms the change: the string is now -rw-rw-r--, so the owner and group can still read and write the file, but everyone else can only read it.

Change file permissions on a hidden file

The hidden file .project_x.txt holds the research team's archived project. It currently has -rwxrwxrwx permissions — full read, write, and execute access for everyone. Per the scenario, this file should have no write access for anyone, while the owner and group should still be able to read it. I used chmod with numeric (octal) mode to set this precisely:

```
researcher2@Linux-Instance:~/projects$ chmod 440 .project_x.txt
researcher2@Linux-Instance:~/projects$ ls -l .project_x.txt
-r--r----- 1 researcher2 research 128 Aug 20 09:10 .project_x.txt
```

In octal mode, each digit sets the full permission set for one group at once: the first digit is the owner, the second is the group, and the third is other. `4` corresponds to read-only (r--), so `440` sets read-only access for the owner and the group, and removes all permissions — including execute, which a text file shouldn't have anyway — for other. The resulting string, -r--r-----, matches exactly what the scenario asks for: read access for the owner and group, and nothing for anyone else. A file being “hidden” (its name starting with a dot) only affects whether it shows up in a normal directory listing — it has no bearing on its permissions, which is why .project_x.txt still needed to be explicitly locked down like any other file.

Change directory permissions

The drafts directory and everything inside it belongs to researcher2, and per the scenario, only researcher2 should be able to access it — not the rest of the research group, and not anyone else. Its current permissions, drwxrwxr-x, give the group read and execute access and let everyone else browse and enter it too. I used chmod to restrict it to the owner only:

```
researcher2@Linux-Instance:~/projects$ chmod 700 drafts
researcher2@Linux-Instance:~/projects$ ls -ld drafts
drwx------ 1 researcher2 research 4096 Aug 19 16:45 drafts
```

Here, octal mode `700` grants the owner full read, write, and execute access (7 = rwx) while setting both the group and other permissions to 0 (---), removing all access for anyone but researcher2. Execute permission on a directory controls whether a user can enter (cd into) it and access its contents, so removing it for group and other — not just read — is what actually prevents them from getting into drafts at all, even if they somehow knew a file name inside it. I used `ls -ld` rather than plain ls -l here so that the command reports on the drafts directory itself rather than listing the files inside it.

Summary

In this activity, I audited the permissions on the researcher2 user's projects directory using ls -la and compared what I found against the research team's access policy. I identified three permission problems: a shared file and a hidden, archived file that both granted write access to users outside the owner and group, and a drafts directory that was accessible to the whole research group instead of just its owner. I used chmod — both symbolic (o-w) and numeric (440, 700) modes — to correct each one: removing unauthorized write access from lab_notes.txt, locking .project_x.txt to read-only for the owner and group, and restricting the drafts directory so only researcher2 can access it. Together, these changes bring the file system's actual permissions back in line with what the organization authorizes, which is the core goal of access control and authorization management.